

Presentation Link:

https://docs.google.com/presentation/d/1fzSlO4qkVNu_w-KCJZflgTlrvQ_V-0lu/edit?usp=sharing&oid=112274446448482078707&rtpof=true&sd=true

Problem 1: Method for Optimal Pairwise Alignment of Circular Sequences

Circular chromosomes present a unique challenge for sequence alignment because they do not have a fixed starting point. In practice, circular DNA sequences are stored as linear strings by introducing a breakpoint. However, different researchers may choose different breakpoints, which causes homologous regions to appear misaligned even when the sequences are highly similar. Standard global alignment algorithms such as Needleman–Wunsch assume linear sequences with consistent ordering and therefore cannot directly account for these breakpoint differences.

To address this issue, we propose a rotation-based alignment approach. The key idea is that circular sequences can be represented by all possible linear rotations of the same sequence. A rotation shifts the sequence by a certain number of base pairs by moving the beginning of the sequence to the end. For example, rotating the sequence *AAATGATGGG* by three positions produces *GGGAAATGAT*. This preserves all biological information while changing the linear representation.

Our method proceeds as follows. Given two sequences, S_i and S_j , we fix S_i as a reference and systematically rotate S_j . For each rotation, we perform a **global alignment using the Needleman–Wunsch algorithm**, which accounts for mismatches and gaps. The alignment is scored using a simple scoring scheme (match = +2, mismatch = -1, gap penalty = -2), allowing the method to tolerate up to ~10% mismatches and small differences in sequence length.

To improve efficiency, we implement a two-pass search strategy. In the first pass, the sequence is rotated in larger increments (10 base pairs) to identify a promising region. In the second pass, the search is refined by testing rotations one base pair at a time within a small window around the best candidate. This approach significantly reduces computation time while still identifying the optimal or near-optimal alignment.

The optimal alignment is defined as the rotation that produces the highest alignment score. Once this rotation is identified, the corresponding alignment is selected as the final global alignment between the two sequences.

Rationale

Circular sequences are invariant to rotation, meaning their biological content does not depend on the starting position. By systematically rotating one sequence, we effectively search for the correct linear representation that aligns homologous regions. Combining this with a global alignment algorithm ensures that mismatches and insertions/deletions are properly accounted for.

Assumptions

- The sequences are circular and represent similar genomic elements
- Breakpoint differences are the main cause of misalignment
- Sequence differences are relatively small ($\leq 10\%$ mismatches)
- A simple scoring scheme is sufficient to capture similarity

Strengths

- Corrects for arbitrary breakpoint differences in circular DNA
- Integrates with standard alignment algorithms
- Handles mismatches and small length differences
- Efficient due to two-pass rotation search

Limitations

- Computational cost increases with sequence length
- Two-pass search is heuristic and may not always guarantee the absolute global optimum
- Progressive alignment (in MSA) is greedy and depends on sequence order
- Assumes sequences are circular without verification

Problem 2: Python Implementation Outline

The proposed method is implemented in Python using a modular structure consisting of several key functions.

Input Handling

- User will give sequence they will be comparing against
- User will give sequence that will be rotating to be compared

Key Functions

1. `needlemanWunsch(seq1, seq2)`

- Implements global alignment using dynamic programming
- Uses a 2D matrix (list of lists) to store alignment scores
- Steps:
 - Initialize scoring matrix
 - Fill matrix using match/mismatch/gap scoring
 - Perform traceback to construct aligned sequences
- Returns:
 - aligned sequence 1
 - aligned sequence 2
 - alignment score

Merit: Guarantees optimal global alignment given a scoring scheme.

2. circularAlignment(seq1, seq2, ...)

- Handles circular breakpoint correction
- Rotates sequence using string slicing:
- Performs alignment for each rotation
- Uses a **two-pass strategy**:
 - Coarse search
 - Fine search (local refinement)
- Tracks:
 - best alignment score
 - best rotation index
 - best aligned sequences

Merit: Efficiently identifies the optimal breakpoint alignment without testing every possible rotation.

3. progressiveMSA(sequences)

- Builds a multiple sequence alignment progressively
- Steps:
 - Select first sequence as reference
 - Align each new sequence to reference
 - Insert gaps into previously aligned sequences
- Uses lists and string manipulation

Merit: Simple and computationally efficient for small datasets.

4. printMSA()

- Outputs the final aligned sequences in readable format

Overall program flow:

- Read user input
- Select reference sequence
- Rotate and align sequences
- Update alignment progressively
- Output final MSA

Key Python Concepts Used

- Lists (alignment matrix and sequence storage)
- Strings (sequence manipulation and rotation)
- Loops (for, while)
- Conditionals (if/else)
- Functions (modular design)
- Dynamic programming

Advantages of This Implementation

- Modular and easy to understand
- Efficient due to two-pass search strategy
- Scalable to multiple sequences
- Uses well-established algorithms

Problem 3: Figure Description

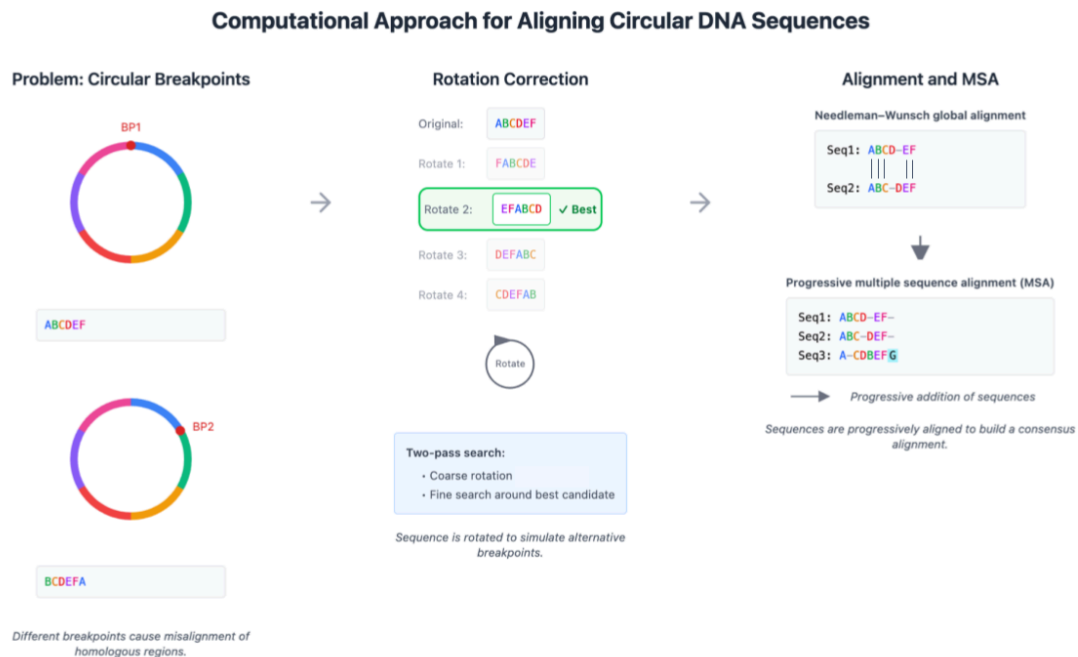


Figure 1. Computational approach for aligning circular DNA sequences

The figure illustrates the three main stages of the alignment process.

- **(A) Circular Breakpoints:**
Circular genomes are linearized at different breakpoints, causing homologous regions to appear misaligned when sequences are compared directly.
- **(B) Rotation Correction:**
One sequence is systematically rotated to simulate alternative breakpoint positions. A two-pass search strategy (coarse and fine rotation) is used to efficiently identify the rotation that produces the highest alignment score.
- **(C) Alignment and MSA:**
After identifying the optimal rotation, sequences are globally aligned using the Needleman–Wunsch algorithm. Additional sequences are progressively aligned to construct a multiple sequence alignment (MSA).

References:

Durbin, R., De Sanctis, B., & Blumer, M. (2023, September 13). *Rotate: A command-line program to rotate circular DNA sequences to start at a given position or string*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11056001/>

Richarddurbin. (n.d.). *Richarddurbin/Rotate: Simple Program to rotate a circular sequence to start at a given position or string*. GitHub.

<https://github.com/richarddurbin/rotate>